Designing a Trace-Driven Scheduling Method for Microservices in Heterogeneous Cloud-Edge Environments

Abstract

Microservice placement in heterogeneous cloud-edge environments is challenging because service dependencies, workload patterns, and node-level resource contention jointly affect end-to-end latency. Existing scheduling methods often search within broad candidate spaces, while trace-based diagnosis is mainly used after performance degradation occurs. This paper proposes a trace-driven scheduling support method that learns service dependency and bottleneck-related information from runtime traces and converts it into prior knowledge for candidate placement generation. The method combines service-level dependency inference with node-level performance information to narrow the placement search space and rank candidate deployments before runtime scheduling. We implement the workflow on a Kubernetes-based microservice testbed and compare it with baseline placement strategies using latency and resource-utilization metrics. The goal is to reduce scheduling decision overhead while preserving interpretable support information for placement optimization.

1 Introduction

Modern microservice applications are increasingly deployed across heterogeneous cloud-edge environments, where nodes differ in computing capacity, resource availability, and communication delay. In such environments, service placement has a direct impact on end-to-end latency and resource utilization. However, a request in a microservice system usually involves multiple dependent service calls, including sequential execution, fan-out patterns, and repeated invocations of performance-critical services. Therefore, scheduling decisions cannot be made effectively by considering only the resource demand of each service in isolation.

Existing studies have addressed microservice performance from several directions, including placement optimization, runtime elasticity, and trace-based diagnosis. Placement optimization methods search for better deployments under resource constraints, while elasticity mechanisms adjust replicas or resources after workload changes are observed. Trace-based diagnosis methods can identify bottlenecks and performance anomalies from runtime traces. However, trace-derived knowledge is often used mainly for post-hoc analysis, rather than being converted into prior knowledge for scheduling decisions. As a result, scheduling may still rely on a large candidate space or insufficient information about service dependencies and performance-impact propagation.

This paper proposes a trace-driven scheduling support method for microservice placement in heterogeneous cloud-edge environments. The key idea is to learn dependency and bottleneck-related information from runtime traces and use it to guide placement candidate generation and filtering. Instead of performing expensive online optimization, the proposed method aims to provide lightweight scheduling support by narrowing the candidate space and prioritizing placements that are more consistent with observed execution behavior.

The contributions of this paper are threefold. First, we present a workflow that converts runtime traces into scheduling support knowledge. Second, we introduce a trace-guided candidate adjustment strategy based on service dependency and bottleneck hints. Third, we conduct an early-stage evaluation to examine the feasibility of using trace-derived knowledge for microservice placement support.

2 Related Work

Microservice performance management has been studied from multiple perspectives because deployment decisions, runtime resource allocation, and failure diagnosis are closely related in distributed applications. In particular, service placement determines where each microservice instance is executed, auto-scaling adjusts replicas or resource limits according to workload changes, and trace-based diagnosis analyzes runtime behavior to identify bottlenecks or root causes after performance degradation occurs. These techniques address different stages of the system runtime, but they share a common goal: improving application-level performance under limited and dynamic infrastructure resources.

2.1 Microservice Placement and Scheduling

Microservice placement and scheduling have been studied to improve application performance in heterogeneous cloud, edge, and fog environments. Pallewatta et al. [7] proposed a decentralized placement policy for microservice-based IoT applications in heterogeneous fog environments. Polaris Scheduler [8] further shows that edge scheduling should consider not only resource requirements but also network QoS.

Recent studies also incorporate topology and runtime information into scheduling decisions. MOTAS [5] uses microservice and cluster topologies for topology-aware scheduling, and introduces distributed trace analysis to handle dynamic QoS violations. NetMARKS [9] uses dynamic network metrics collected through Istio Service Mesh to improve Kubernetes pod scheduling.

These studies show that microservice scheduling requires information beyond static CPU or memory availability, including service topology, network state, and runtime telemetry. However, their main focus is scheduler design, topology-aware placement, or network-aware node scoring. Even when traces are used, they are mainly integrated into scheduling frameworks for topology reconstruction or QoS handling.

In contrast, this work focuses on extracting dependency and bottleneck-related hints from runtime traces and converting them into support for resource and deployment adjustment. Rather than directly replacing the Kubernetes scheduler, the proposed method provides trace-derived prior knowledge to narrow adjustment candidates and make resource reconfiguration more consistent with observed execution behavior.

2.2 Autoscaling and Resource Reconfiguration

Auto-scaling is widely used to adapt microservice deployments to workload changes. Beyond simple metric-based scaling, recent studies attempt to decide which services should be adjusted. Microscaler [11] uses online learning to identify services that require scaling and determine suitable service sizes under SLA constraints.

Auto-scaling has also been extended from horizontal scaling to hybrid resource adjustment. Horn et al. [4] combine horizontal and vertical scaling for multi-objective auto-scaling in Kubernetes clusters. However, such service-level adjustment may still be insufficient when the performance impact of one service propagates through dependent services.

This limitation motivates coordinated and dependency-aware auto-scaling. Chamulteon [2] shows that scaling services independently can cause bottleneck shifting or oscillation. Graph-PHPA [6] further uses graph-based learning to capture dependency-aware workload behavior for proactive horizontal pod auto-scaling.

These studies suggest that auto-scaling decisions should consider service interactions and bottleneck propagation, not only local metrics. Based on this observation, this work uses runtime traces to extract dependency and bottleneck hints for resource and replica reconfiguration. Rather than designing a new auto-scaling controller, the proposed method provides trace-derived prior knowledge to guide adjustment candidates under shared-resource constraints.

2.3 Trace-Based RCA and Diagnosis-Guided Reconfiguration

Trace-based RCA has been studied to identify services or resources responsible for microservice performance problems. Sage [3] is a representative ML-driven RCA system that uses RPC-level traces, causal modeling, and counterfactual reasoning to locate root causes of QoS violations. It also applies corrective actions, showing that RCA results can be connected with resource or deployment changes.

However, RCA-based correction is mainly triggered by observed performance problems. Even in Sage, corrective actions focus on services or resources related to detected QoS violations. When such results are directly used as an adjustment policy, the response may remain local and reactive.

This limitation is related to findings from autoscaling studies. Chamulteon [2] shows that independently scaling services can cause bottleneck shifting or oscillation. Smart HPA [1] further points out that some microservices may be overprovisioned while others are underprovisioned, and introduces resource exchange among microservices.

These studies suggest that diagnosis-guided adjustment should consider not only problematic services, but also less critical or overprovisioned services whose reserved resources can be redistributed. Based on this observation, this work uses trace-derived dependency and bottleneck hints as intermediate knowledge for resource and replica reconfiguration under shared-resource constraints.

2.4 Learning-Based Scheduling and Resource Management

Learning-based methods have recently been applied to microservice scheduling, auto-scaling, and resource management. Sinan [12] uses tracing data and ML models to estimate the impact of resource allocation on end-to-end tail latency. CoScal [10] applies reinforcement learning to combine horizontal scaling, vertical scaling, and brownout for balancing response time and cost.

These methods show that ML and RL can support more adaptive resource management than simple threshold-based control. Graph-PHPA [6] further uses LSTM-GNN models for proactive horizontal pod auto-scaling by considering graphical dependencies among microservices, indicating that learning-based methods can also incorporate dependency-aware behavior.

However, fully learning-based control requires sufficient training data, a well-defined action space, and continuous feedback from the deployment environment. Its reliability

also depends on whether future workload patterns, resource-contention situations, and long call-chain effects are covered by the learned model.

Therefore, this work does not replace RCA, auto-scaling, and scheduling with a single end-to-end learning controller. Instead, it extracts trace-derived dependency and bottleneck hints as interpretable intermediate knowledge for resource and replica reconfiguration, while leaving room for future integration with learning-based methods.

3 Methodology

3.1 Overview

Figure **??** shows the overall workflow of the proposed trace-driven scheduling support framework. The main idea is to convert runtime traces and cluster metrics into reusable support information for deployment adjustment. Instead of relying only on static resource requests, exhaustive candidate search, or post-hoc correction, the framework extracts dependency and bottleneck-related hints from observed executions and uses them to guide candidate generation and filtering.

The framework consists of three connected stages: state inference, suggestion generation, and scoring/filtering. In the first stage, runtime traces and metrics are analyzed to infer service-level and node-level states. In the second stage, the inferred information is used to generate candidate adjustment strategies. In the third stage, these candidates are scored and filtered according to their expected resource and latency impact. The retained strategies can then be reused by the scheduling proposer in later deployment adjustment cycles.

3.2 Service- and Node-State Inference

The first stage converts raw runtime observations into scheduling-related priors. Instead of using traces only for post-hoc diagnosis, the framework treats traces, task status, cluster status, and node-side metrics as reusable evidence for deployment adjustment. These inputs describe both how requests are executed and under what resource conditions they are executed. Therefore, this stage analyzes runtime observations from two complementary views: the service layer and the node layer.

On the service side, the framework analyzes trace-derived call chains to infer coarse service dependencies and frequent execution patterns. It observes which services appear together, which parent-child service pairs repeatedly occur, and which services are frequently involved in long or latency-sensitive call paths. Span-level latency information is also used to estimate bottleneck-related services, preferably by focusing on a service's own latency contribution rather than assigning all downstream latency to its parent. As a result, the service layer produces hints such as frequent services, strong dependency pairs, and bottleneck-related services for later adjustment.

However, service-level inference alone is not sufficient for deployment adjustment. A service may be important in the trace structure, but increasing its CPU or replicas can still worsen contention if the node is already heavily loaded. Therefore, the node layer uses runtime metrics and cluster status to represent resource conditions, including CPU and memory utilization, waiting time, and contention-related signals. This layer provides the performance context needed to judge whether a service-level adjustment is feasible under the current resource environment.

By combining the two layers, the framework obtains both execution-structure information and resource-condition information. The service layer answers which services and dependencies are important in observed executions, while the node layer answers under what resource conditions candidate adjustments should be compared. These complementary priors are then passed to the Suggestion Generator, where they are used to guide resource, replica, and future placement adjustment candidates.

3.3 Trace-Guided Suggestion Generation

The second stage generates candidate adjustment strategies from the inferred service- and node-level priors. Instead of enumerating all possible deployments, the Suggestion Generator uses trace-derived dependency hints, bottleneck-related services, and observed deployment behavior to narrow the candidate space. The purpose of this stage is to transform runtime observations into actionable candidates for resource, replica, and future placement adjustment.

The generated candidates can include several types of changes. For latency-sensitive or bottleneck-related services, the generator may propose increasing CPU allocation or adding replicas. For services that appear less critical in the observed call chains, it may propose releasing reserved resources. In addition, dependency-aware hints allow related services to be considered together, so that an adjustment is not made only for an isolated service but for the call-chain segment in which the service repeatedly appears.

In a multi-node setting, the generator can also reuse observed deployment patterns to create placement-refinement candidates. For example, suppose a trace shows that replica A_1 runs on node 1, replicas B_1 and B_2 run on node 2, and replicas C_1 and B_3 run on node 3. If requests routed to B_3 repeatedly become long-tail requests, the framework treats B_3 as a placement-sensitive candidate rather than simply increasing resources for service B.

The generator then searches for similar execution or deployment patterns in previous traces. Suppose another observed pattern from a comparable call chain places replica X_1 on node 1, replicas Y_1, Y_2, and Y_3 on node 2, and replica Z_1 on node 4. If this pattern does not show the same long-tail behavior, node 2 or node 4 can be considered as possible alternative locations for the problematic replica. Therefore, the generator creates candidate strategies such as moving B_3 from node 3 to node 2 or node 4. These candidates are not directly applied; they are passed to the scoring and filtering stage, where their expected latency benefit and resource-contention risk are evaluated.

3.4 Scoring and Filtering

The third stage ranks candidate strategies before they are retained or applied. The scoring function is designed to compare the expected benefit and risk of each candidate, rather than to prove global optimality. Each candidate c is scored as:

$$Score(c) = B(c) - P(c), \tag{1}$$

where $B(c)$ represents the expected benefit of the candidate and $P(c)$ represents its expected cost or risk. In this work, the benefit mainly comes from improving services that are important in the observed traces, such as frequently invoked services, bottleneck-related services, or services on latency-sensitive call paths. The penalty mainly comes from additional resource pressure, replica expansion, or contention risk.

More specifically, the benefit and penalty are computed with dynamic coefficients:

$$B(c) = \sum_{s \in S_c} I_s H_s \cdot G_s(c), \tag{2}$$

$$P(c) = \sum_{n \in N_c} \rho_n \cdot C_n(c) + \sum_{s \in S_c} \eta_s \cdot R_s(c). \tag{3}$$

Here, S_c is the set of services affected by candidate c, and N_c is the set of nodes affected by the candidate. I_s denotes the trace-derived importance of service s, and H_s denotes its bottleneck tendency. $G_s(c)$ represents the expected gain of adjusting service s. On the penalty side, $C_n(c)$ represents the additional resource load introduced to node n, and ρ_n is a node-pressure coefficient. $R_s(c)$ represents the cost of replica or resource expansion for service s, and η_s controls how strongly the method penalizes unnecessary expansion.

For example, when candidate c increases the CPU allocation of service s, the expected gain can be written as:

$$G_s^{cpu}(c) = \phi(\Delta CPU_s), \tag{4}$$

Table 1: Cluster environment.

Item	Setting
Cluster type	Single-node minikube
CPU capacity	8 cores
Memory capacity	16 GB
Kubernetes client	v1.30.0
Kubernetes server	v1.29.2
Kustomize	v5.0.4

where ΔCPU_s is the increased CPU allocation and $\phi(\cdot)$ is a diminishing-return function. In this paper, we use a saturating form:

$$\phi(x) = 1 - e^{-\alpha x}, \tag{5}$$

where α controls how quickly the benefit saturates. This means that increasing CPU for an under-provisioned bottleneck service can receive a high reward, while adding more CPU to an already well-provisioned service provides a smaller marginal benefit.

The node-pressure coefficient is used to avoid aggressive resource expansion on already loaded nodes:

$$\rho_n = \max(0, U_n - \theta)^2, \tag{6}$$

where U_n is the current utilization of node n, and θ is the resource-pressure threshold. When the node has sufficient remaining resources, U_n is below the threshold and the penalty remains small. When the node is already resource-constrained, the penalty increases sharply. Therefore, the same CPU increase can be acceptable on a lightly loaded node but strongly penalized on a heavily loaded node.

Candidates with low scores are filtered out, while high-scoring candidates are retained as reusable adjustment strategies. This scoring design prevents the framework from blindly increasing resources or replicas, and allows trace-derived importance, bottleneck tendency, and current resource pressure to be considered together.

4 Experiment

4.0.1 Cluster Environment. The experiment is conducted on a single-node minikube cluster. The cluster provides 8 CPU cores and 16 GB of memory. Since all microservice replicas run on the same node, this setting creates a constrained shared-resource environment and allows us to focus on resource and replica reconfiguration. Table 1 summarizes the cluster environment.

4.0.2 Application and Workload. The application consists of 12 microservices. Service s0 is used as the entry service and returns a string response after receiving downstream responses. The remaining services generate CPU-bound workload by computing digits of π with floating-point operations. Most services use the default workload size of 1000 digits,

Table 2: Application and workload configuration.

Item	Setting
Services	12 microservices
Entry service	$s0$
Workload type	CPU-bound π computation
Default workload	1000 digits
Heavy service	$s2$, 5000 digits
Medium service	$s4$, 2500 digits
Request types	3 types
Requests per type	20
Total requests	60
Concurrency	6 concurrent requests
Timeout	5 minutes

Table 3: Compared deployment strategies.

Strategy	Pods	Policy
Baseline	24	Two replicas per service; no service-specific tuning.
Proposed	24	Trace-guided reconfiguration of replicas and resource requests.
RCA-based	Increased	Increase resources or replicas for heavy and hotspot services only.

while $s2$ is configured as a heavy service with 5000 digits and $s4$ as a medium service with 2500 digits.

Each run sends three request types, with 20 requests for each type, resulting in 60 requests in total. The concurrency level is fixed at six, meaning that at most six requests are executed at the same time. This setting is chosen to keep the experiment stable under the 8-core testbed; higher concurrency caused many timeout responses and would contaminate latency comparison. The request timeout is set to 5 minutes.

4.0.3 Compared Strategies and Metrics. We compare three deployment strategies. The baseline uses two replicas for each service, resulting in 24 workload pods, and applies no service-specific resource tuning. The proposed strategy keeps the total number of workload pods at 24, but changes replica distribution and resource requests according to trace-derived dependency and bottleneck hints. The RCA-based strategy represents reactive experience-driven tuning: after heavy and hotspot services are identified, their replicas and resources are increased, while normal workload services are not actively reduced or redistributed.

All strategies use the same application code, request mix, timeout setting, and cluster environment. The $p95$ latency is calculated from successful client-side request durations. Timeout or failed requests are excluded from the $p95$ calculation to avoid mixing availability failures with latency distribution. CPU utilization is collected from Prometheus with a 30-second sampling interval during the execution window.

4.1 Experiment Result

Table 4 summarizes the end-to-end latency results. The proposed strategy achieves the lowest $p50$, $p95$, and standard deviation among the three strategies, although its mean latency is close to the baseline. This indicates that the main benefit of the proposed method is not a large reduction in average latency, but improved tail latency and more stable execution behavior.

Table 4: Latency comparison of the three strategies.

Strategy	Mean	$p50$	$p95$	Std. dev.	Max
Proposed	130.396	124.456	146.870	11.344	159.685
Baseline	129.850	128.593	151.190	12.544	161.559
RCA-based	131.053	128.743	160.324	16.610	184.440

Table 5 shows that the proposed strategy mainly improves stability on trace-02 and trace-03. For trace-02, the proposed strategy reduces $p95$ latency from 150.833 s in the baseline and 160.607 s in the RCA-based strategy to 125.713 s. For trace-03, it also reduces $p95$ latency compared with both baseline and RCA-based tuning. However, for trace-01, the baseline achieves slightly lower $p95$ latency than the proposed strategy. This suggests that the proposed method is effective for reducing long-tail behavior in some request paths, but its scoring function still needs refinement for paths similar to trace-01.

Figure 1 shows the latency distribution of the three strategies. Compared with the RCA-based strategy, the proposed strategy has a lower upper percentile range and fewer extreme long-tail requests. The RCA-based strategy shows the widest high-latency range, with the slowest request reaching 184.44 s. This indicates that simply increasing resources or replicas for heavy and hotspot services can introduce unstable tail behavior under the shared-resource constraint. In contrast, the proposed strategy reduces the $p95$ latency and standard deviation, suggesting that trace-guided reconfiguration improves execution stability rather than only reducing average latency.

Figure 2 compares the overall CPU utilization during the execution window. The three strategies show similar CPU utilization trends during the main execution period, and the proposed strategy does not obtain its latency improvement by consuming clearly more CPU resources. This supports the interpretation that the improvement mainly comes from resource and replica reconfiguration guided by trace-derived hints, rather than from simply expanding resource usage.

4.2 Analysis

Table 4 shows that the proposed method does not greatly reduce mean latency compared with the baseline. The mean latency is 130.396 s for the proposed method and 129.850 s

Table 5: Per-trace latency comparison.

Strategy	Trace	Mean	$p50$	$p95$	Std. dev.
Proposed	trace-01	145.345	144.184	155.434	4.778
	trace-02	122.158	121.971	125.713	2.272
	trace-03	123.684	123.420	126.666	4.182
Baseline	trace-01	139.518	140.384	153.208	6.770
	trace-02	124.277	121.131	150.833	12.184
	trace-03	125.754	122.857	144.270	12.065
RCA-based	trace-01	143.428	139.451	171.141	13.237
	trace-02	129.642	127.141	160.607	17.393
	trace-03	120.088	117.380	140.842	9.439

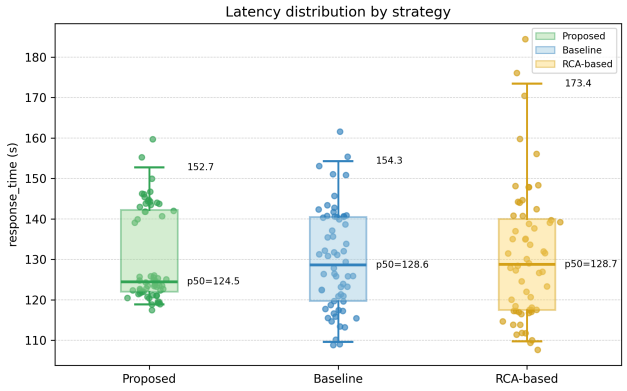

Figure 1: Request latency distribution for the three strategies. Dots represent individual successful requests, the box shows the interquartile range, the center line indicates the median, and the whiskers show the central 95% percentile range.

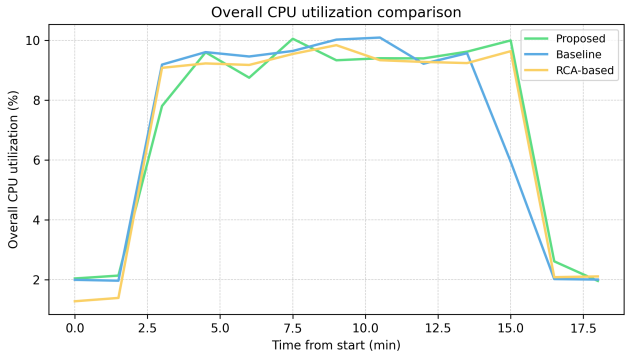

Figure 2: Overall CPU utilization comparison. The CPU series exported from Grafana are aggregated into an overall utilization curve and normalized from the start of each run.

for the baseline. This is expected in the current single-node setting, where 12 services share limited 8-core resources and the room for reducing total execution time is small. Therefore, the main effect of the proposed method should be evaluated through tail latency and stability rather than mean latency alone.

The proposed method achieves the lowest $p50$, $p95$, and standard deviation among the three strategies. Its $p95$ latency is 146.870 s, compared with 151.190 s for the baseline and 160.324 s for the RCA-based strategy. Its standard deviation is also lower, 11.344 s compared with 12.544 s and 16.610 s. As shown in Fig. 1, the proposed method has a more compact latency distribution, while the RCA-based strategy shows a wider upper range and stronger long-tail behavior.

The per-trace results in Table 5 show that the improvement mainly comes from trace-02 and trace-03. For trace-02, the proposed method reduces $p95$ latency from 150.833 s in the baseline and 160.607 s in the RCA-based strategy to 125.713 s. For trace-03, it reduces $p95$ latency to 126.666 s. However, for trace-01, the baseline still achieves a slightly lower $p95$ latency. This suggests that the current scoring function is effective for some call-chain patterns, but still needs refinement for others.

Fig. 2 shows that the three strategies have similar CPU utilization trends during the main execution period. Thus, the proposed method does not obtain better stability by clearly consuming more CPU resources. Instead, the result suggests that trace-derived dependency and bottleneck hints help redistribute resources and replicas more effectively under shared-resource constraints. In contrast, the RCA-based strategy increases resources or replicas for selected heavy and hotspot services without reducing less critical services, which may introduce additional contention and increase tail latency.

4.3 Discussion

This study has several limitations. First, the current evaluation is conducted on a single-node minikube cluster with 8 CPU cores. Therefore, the experiment mainly validates resource and replica reconfiguration under shared-resource constraints, while inter-node placement, node affinity, and network-aware scheduling are left for future multi-node experiments.

Second, practical deployment requires careful control of update frequency. Since the proposed method adjusts resources and replicas during operation, frequent reconfiguration may introduce management overhead, service disturbance, or unstable feedback effects. A cooldown interval, update threshold, or hysteresis mechanism is needed to prevent small fluctuations from triggering unnecessary adjustments.

Third, the method relies on previously observed traces. If a rarely used service suddenly becomes a hotspot due to a new call chain or workload shift, the trace-derived hints may

underestimate its importance. Although the method reserves resources according to frequency and bottleneck tendencies, sudden changes in request patterns still require online monitoring and fallback mechanisms. Future work will address these issues by extending the framework to multi-node environments and studying adaptive update policies.

References

[1] Hussain Ahmad, Christoph Treude, Markus Wagner, and Claudia Szabo. 2024. Smart HPA: A Resource-Efficient Horizontal Pod Auto-Scaler for Microservice Architectures. In *Proceedings of the IEEE 21st International Conference on Software Architecture (ICSA '24)*. IEEE, 46–57. doi:10.1109/ICSA59870.2024.00013

[2] André Bauer, Veronika Lesch, Laurens Versluis, Alexey Ilyushkin, Nikolas Herbst, and Samuel Kounev. 2019. Chamulteon: Coordinated Auto-Scaling of Micro-Services. In *Proceedings of the 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS '19)*. IEEE, 2015–2025. doi:10.1109/ICDCS.2019.00199

[3] Yu Gan, Mingyu Liang, Sundar Dev, David Lo, and Christina Delimitrou. 2021. Sage: Practical and Scalable ML-Driven Performance Debugging in Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 135–151. doi:10.1145/3445814.3446700

[4] Angelina Horn, Hamid Mohammadi Fard, and Felix Wolf. 2022. Multi-objective Hybrid Autoscaling of Microservices in Kubernetes Clusters. In *Euro-Par 2022: Parallel Processing (Lecture Notes in Computer Science, Vol. 13440)*. Springer, 233–250. doi:10.1007/978-3-031-12597-3_15

[5] Xin Li, Junsong Zhou, Xin Wei, Dawei Li, Zhuzhong Qian, Jie Wu, Xiaolin Qin, and Sanglu Lu. 2023. Topology-Aware Scheduling Framework for Microservice Applications in Cloud. *IEEE Transactions on Parallel and Distributed Systems* 34, 5 (2023), 1635–1649. doi:10.1109/TPDS.2023.3238751

[6] Hoa X. Nguyen, Shaoshu Zhu, and Mingming Liu. 2022. Graph-PHPA: Graph-Based Proactive Horizontal Pod Autoscaling for Microservices Using LSTM-GNN. In *2022 IEEE 11th International Conference on Cloud Networking (CloudNet '22)*. IEEE. doi:10.1109/CloudNet55617.2022.9978781

[7] Samodha Pallewatta, Vassilis Kostakos, and Rajkumar Buyya. 2019. Microservices-Based IoT Application Placement within Heterogeneous and Resource Constrained Fog Computing Environments. In *Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing (UCC '19)*. Association for Computing Machinery, New York, NY, USA, 71–81. doi:10.1145/3344341.3368800

[8] Thomas Pusztai, Stefan Nastic, Andrea Morichetta, Víctor Casamayor Pujol, Philipp Raith, Schahram Dustdar, Deepak Vij, Ying Xiong, and Zhaobo Zhang. 2022. Polaris Scheduler: SLO- and Topology-Aware Microservices Scheduling at the Edge. In *2022 IEEE/ACM 15th International Conference on Utility and Cloud Computing (UCC '22)*. IEEE, 61–70. doi:10.1109/UCC56403.2022.00017

[9] Łukasz Wojciechowski, Krzysztof Opasiak, Jakub Latusek, Maciej Wereski, Victor Morales, Taewan Kim, and Moonki Hong. 2021. NetMARKS: Network Metrics-AwaRe Kubernetes Scheduler Powered by Service Mesh. In *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*. IEEE, 1–9. doi:10.1109/INFOCOM42981.2021.9488670

[10] Minxian Xu, Chenghao Song, Shashikant Ilager, Sukhpal Singh Gill, Juanjuan Zhao, Kejiang Ye, and Cheng-Zhong Xu. 2022. CoScal: Multifaceted Scaling of Microservices With Reinforcement Learning. *IEEE Transactions on Network and Service Management* 19, 4 (2022), 3995–4009. doi:10.1109/TNSM.2022.3210211

[11] Guangba Yu, Pengfei Chen, and Zibin Zheng. 2019. Microscaler: Automatic Scaling for Microservices with an Online Learning Approach. In *2019 IEEE International Conference on Web Services (ICWS '19)*. IEEE, 68–75. doi:10.1109/ICWS.2019.00023

[12] Yanqi Zhang, Weizhe Hua, Zhuangzhuang Zhou, G. Edward Suh, and Christina Delimitrou. 2021. Sinan: ML-Based and QoS-Aware Resource Management for Cloud Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 167–181. doi:10.1145/3445814.3446693